



V1.0.20260615

Digital Signal Processing 2

FS 2026 – Dr. Heinz Mathis

Autoren: Gian Marco Näf

<https://github.com/Giamo08/Zusammenfassungen>

1 Digital Filter Realizations (Chapter 7)

1.1 Equivalent Descriptions of Digital Filters

Ein diskretes LTI-System kann äquivalent beschrieben werden durch:

- Impulsantwort $h(n)$
- Faltung: $y(n) = h(n) * x(n)$
- Transferfunktion $H(z)$
- Frequenzgang $H(e^{j\omega})$
- I/O-Differenzgleichung
- Blockdiagramm (Realisierungsform)

Zusammenhang:

$$H(z) = \sum_{n=-\infty}^{\infty} h(n)z^{-n}$$

$$H(e^{j\omega}) = H(z)|_{z=e^{j\omega}}$$

Designablauf:

1. Spezifikation (Passband, Stopband, Ordnung)
2. Bestimmung von $H(e^{j\omega})$
3. Approximation $\Rightarrow$ rationales $H(z)$
4. Wahl einer Realisierungsstruktur

Wichtig: Alle Realisierungsformen besitzen dieselbe Transferfunktion, unterscheiden sich jedoch in:

- Speicherbedarf
- Anzahl Operationen
- Numerischer Robustheit

1.2 Transfer Function

Allgemeine rationale Form:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1z^{-1} + \dots + b_Lz^{-L}}{1 + a_1z^{-1} + \dots + a_Mz^{-M}} \quad (1)$$

Ordnung des Systems:

$$\text{ord}(H) = M$$

Pole:

$$D(z) = 0$$

Zeros:

$$N(z) = 0$$

Stabilitätsbedingung (BIBO):

$$|p_i| < 1$$

1.3 Herleitung der I/O-Differenzgleichung

Aus

$$H(z) = \frac{Y(z)}{X(z)}$$

folgt:

$$\left(1 + \sum_{k=1}^M a_k z^{-k}\right) Y(z) = \left(\sum_{k=0}^L b_k z^{-k}\right) X(z)$$

Inverse z -Transformation:

$$y(n) + \sum_{k=1}^M a_k y(n-k) = \sum_{k=0}^L b_k x(n-k)$$

Isolieren von $y(n)$:

$$y(n) = -\sum_{k=1}^M a_k y(n-k) + \sum_{k=0}^L b_k x(n-k)$$

Interpretation:

- $b_k \Rightarrow$ Feedforward (Zeros)
- $a_k \Rightarrow$ Feedback (Poles)
- Vorzeichenwechsel bei a_k

1.4 Impulsantwort

Für $x(n) = \delta(n)$:

$$h(n) = y(n)$$

Rekursive Berechnung:

$$h(n) = -\sum_{k=1}^M a_k h(n-k) + b_n$$

mit

$$b_n = \begin{cases} b_n & 0 \leq n \leq L \\ 0 & n > L \end{cases}$$

Initialbedingungen:

$$h(n) = 0 \quad \text{für } n < 0$$

Spezialstruktur 1 – z^{-N}

Da

$$H(z) = \frac{B(z)}{1 - z^{-N}}$$

$$\frac{1}{1 - z^{-N}} = \sum_{k=0}^{\infty} z^{-kN}$$

ergibt sich:

$$h(n) = h(n-N) + b(n)$$

$\Rightarrow$ Impulsantwort ist periodisch mit Periode N .

1.5 Direct Form I

Direkte Umsetzung der Differenzgleichung.

Struktur:

- Eine Delay-Kette für $x(n)$
- Eine Delay-Kette für $y(n)$

Speicherbedarf:

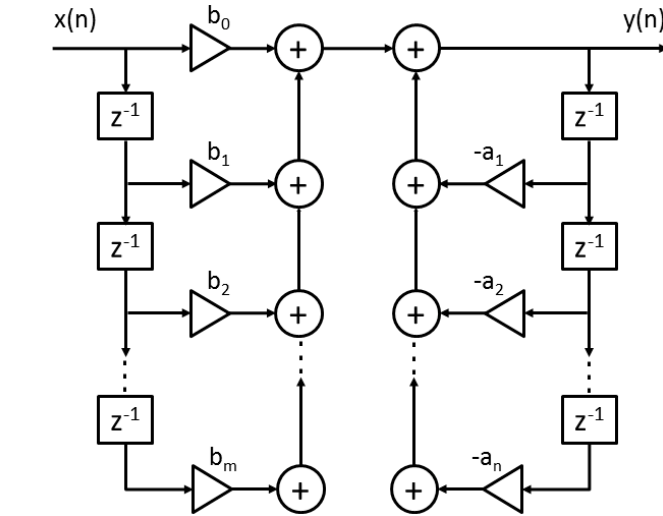
$$L + M$$

Algorithmus:

1. Berechne

$$y(n) = -\sum_{k=1}^M a_k y(n-k) + \sum_{k=0}^L b_k x(n-k)$$

2. Zustände in umgekehrter Reihenfolge aktualisieren



Nachteil:

- Zwei getrennte Delay-Ketten
- Höherer Speicherbedarf

1.6 Direct Form II (Canonical Form)

Idee: Gemeinsame Nutzung der Delay-Elemente.

Zerlegung:

$$H(z) = N(z) \cdot \frac{1}{D(z)}$$

Zustandsgleichung:

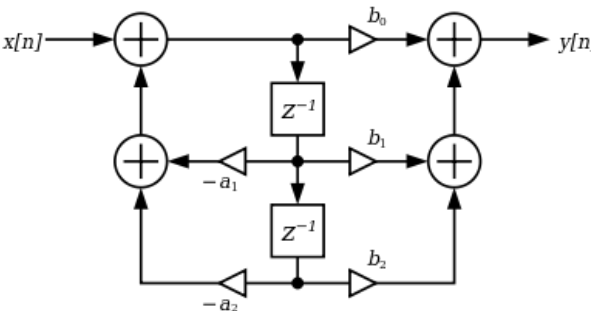
$$w(n) = x(n) - \sum_{k=1}^M a_k w(n-k)$$

Ausgang:

$$y(n) = \sum_{k=0}^L b_k w(n-k)$$

Speicherbedarf:

$$\max(L, M)$$



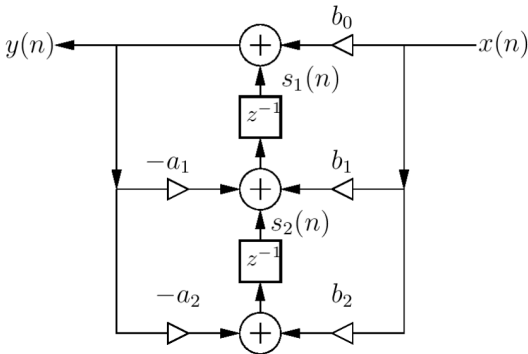
Vorteil:

- Minimaler Speicher
- Nachteil:
 - Numerisch empfindlicher bei hoher Ordnung

1.7 Transposed Form

Transformation eines Blockdiagramms:

- Addierer ↔ Verzweigungen
- Signalflossrichtung umkehren
- Eingang und Ausgang vertauschen



Eigenschaft:

$H(z)$ bleibt unverändert

Oft bessere numerische Eigenschaften bei Festkomma.

1.8 Parallel Form

Mittels Partialbruchzerlegung:

$$H(z) = \sum_i \frac{A_i}{1 - p_i z^{-1}}$$

Implementierung:

- Mehrere Teilsysteme
- Ausgang = Summe der Teiloutputs

Vorteil:

- Gute Interpretierbarkeit einzelner Pole

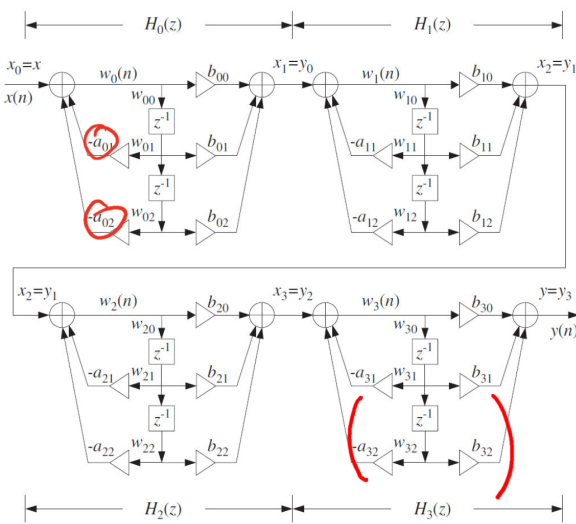
1.9 Cascade Form (Second Order Sections)

Faktorisierung:

$$H(z) = \prod_{i=1}^K \frac{b_{0i} + b_{1i}z^{-1} + b_{2i}z^{-2}}{1 + a_{1i}z^{-1} + a_{2i}z^{-2}}$$

Vorgehen:

1. Nullstellen berechnen
2. Pole berechnen
3. Komplexe konjugierte Paare bilden
4. Reelle Wurzeln paarweise kombinieren
5. Jede Sektion in Canonical Form implementieren



Vorteil:

- Deutlich bessere numerische Stabilität
- Standard bei hoher Ordnung

1.10 DC-Gain

$$H(1) = \frac{\sum_{k=0}^L b_k}{1 + \sum_{k=1}^M a_k}$$

1.11 Hardware Realization and Circular Buffers

Digitale Implementierung:

- Multiply-Accumulate (MAC)
- Koeffizienten in Speicher
- Zustände in RAM

Circular Buffer Prinzip:

- Kein physisches Verschieben der Samples
- Ringstruktur
- Nur Index wird rotiert
- Echtzeitfähig

1.12 Quantization Effects

Endliche Wortlänge führt zu:

- Polverschiebung
- Instabilität
- Rundungsrauschen
- Limit Cycles

Robusteste Struktur:

Second Order Sections

2 Digital Waveform Generators (Chapter 8.1)

2.1 Warum benötigt man digitale Wellenformgeneratoren?

Ein DSP-System kann Signale nicht nur verarbeiten, sondern auch selbst erzeugen.

Typische Anwendungen sind:

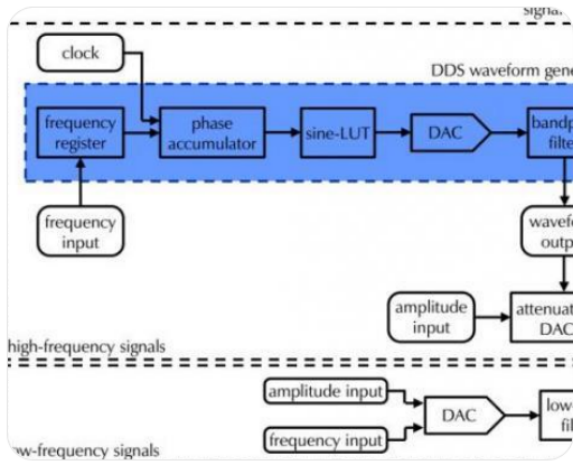
- Musiksynthesizer
- DTMF-Telefon

- Testsignale
- Radar- und Sonarsysteme
- Modulationsverfahren

Ziel ist die Erzeugung eines gewünschten Signals mit möglichst geringem Rechenaufwand.

Grundsätzlich existieren zwei Methoden:

1. Rekursive Generatoren (filterbasiert)
2. Speicherbasierte Generatoren (Wavetable)



2.2 Grundidee digitaler Wellenformgeneratoren

Ziel ist die Erzeugung einer gewünschten diskreten Wellenform $h(n)$.

Man entwirft ein System:

$$H(z)$$

dessen Impulsantwort genau der gewünschten Wellenform entspricht:

$$h(n)$$

Die gewünschte Wellenform kann erzeugt werden als:

1. Impulsantwort eines Filters
 2. Wavetable mit gespeicherter Periode
- Ist das Signal periodisch, genügt es eine einzige Periode zu erzeugen und anschließend zyklisch zu wiederholen.

2.3 Zusammenhang zwischen Polen und Schwingungen

Die wichtigste Erkenntnis dieses Kapitels lautet:

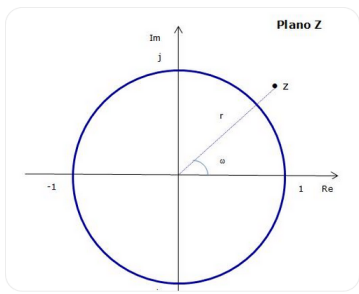
Jeder komplexe Pol erzeugt eine Schwingung.

Ein Pol besitzt die Form

$$p = Re^{j\omega_0}$$

Dabei bestimmt

- der Radius R die Dämpfung
- der Winkel ω_0 die Frequenz



2.4 Sinusgenerator mittels Polstellen auf dem Einheitskreis

for $n = 0, 1, 2, \dots$ do:
 $w_0 = (2R \cos \omega_0) w_1 - R^2 w_2 + \delta(n)$
 $y = (R \sin \omega_0) w_1$
 $w_2 = w_1$
 $w_1 = w_0$

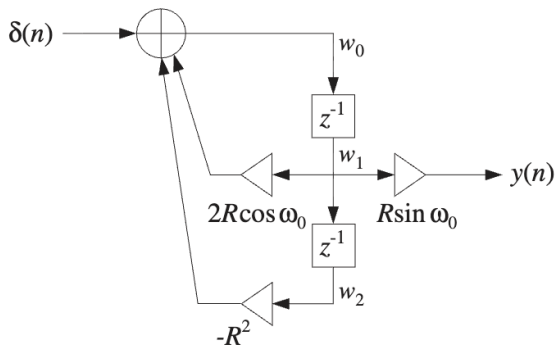


Fig. 8.1.1 Digital sinusoidal generator

Die Pole liegen bei

$$p_{1,2} = R e^{\pm j\omega_0}$$

Die Übertragungsfunktion lautet:

$$H(z) = \frac{R \sin(\omega_0) z^{-1}}{1 - 2R \cos(\omega_0) z^{-1} + R^2 z^{-2}}$$

Impulsantwort:

$$h(n) = R^n \sin(\omega_0 n) u(n)$$

2.5 Warum entstehen Sinus und Kosinus?

Grundlage ist die Euler-Formel:

$$e^{j\omega n} = \cos(\omega n) + j \sin(\omega n)$$

Konjugiert komplexe Pole erzeugen deshalb automatisch:

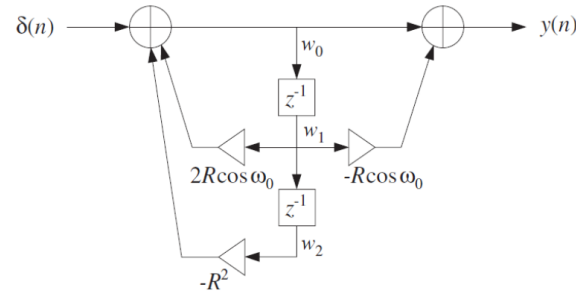
$$R^n \cos(\omega n)$$

und

$$R^n \sin(\omega n)$$

Deshalb basieren nahezu alle digitalen Oszillatoren auf komplex konjugierten Polen.

2.6 Kosinugenerator

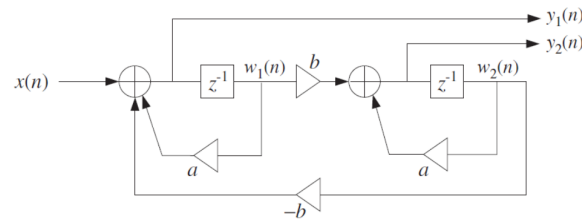


$$H(z) = \frac{1 - R \cos(\omega_0) z^{-1}}{1 - 2R \cos(\omega_0) z^{-1} + R^2 z^{-2}}$$

Impulsantwort:

$$h(n) = R^n \cos(\omega_0 n) u(n)$$

2.7 Gekoppelte Realisierung (Coupled Form)



Definiere:

$$a = R \cos(\omega_0)$$

$$b = R \sin(\omega_0)$$

Dann gilt:

$$R^2 = a^2 + b^2$$

Die Zustandsgrößen erfüllen:

$$w_1(n) = a w_1(n-1) + b w_2(n-1)$$

$$w_2(n) = -b w_1(n-1) + a w_2(n-1)$$

Diese Struktur erzeugt gleichzeitig Sinus und Kosinus.

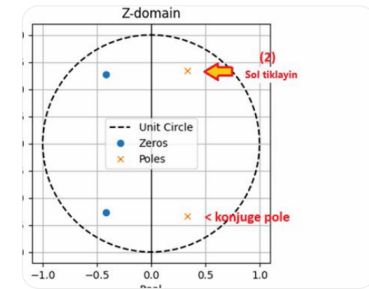
2.8 Polstellenbetrachtung

Für

$$R = 1$$

liegen die Pole exakt auf dem Einheitskreis:

$$p = e^{j\omega_0}$$



Eigenschaften:

- Grenzstabil
- Unendliche Sinusschwingung
- Numerisch empfindlich

2.9 Periodische Wellenformgeneratoren

Gegeben sei eine Periode mit Länge

$$D$$

Dann genügt die Spezifikation

$$h(0), h(1), \dots, h(D-1)$$

Die Grundfrequenz beträgt:

$$f_0 = \frac{f_s}{D}$$

2.10 Filterbasierte Periodenerzeugung

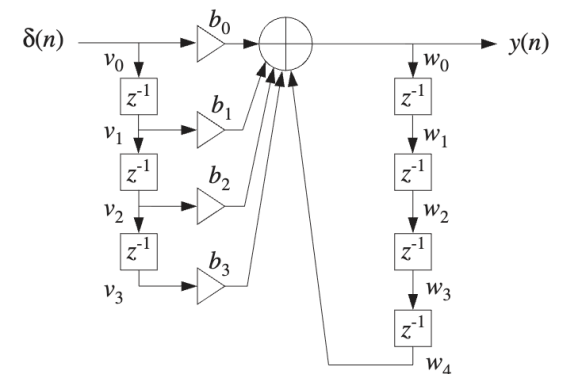


Fig. 8.1.7 Periodic generator in direct form.

Die zentrale Struktur lautet:

$$H(z) = \frac{B(z)}{1 - z^{-D}}$$

Dabei gilt:

$$\frac{1}{1 - z^{-D}} = 1 + z^{-D} + z^{-2D} + z^{-3D} + \dots$$

Interpretation:

- Feedback erzeugt periodische Wiederholung
- FIR-Teil formt die Periode

2.11 Polstellen periodischer Generatoren

Die Pole ergeben sich aus:

$$1 - z^{-D} = 0$$

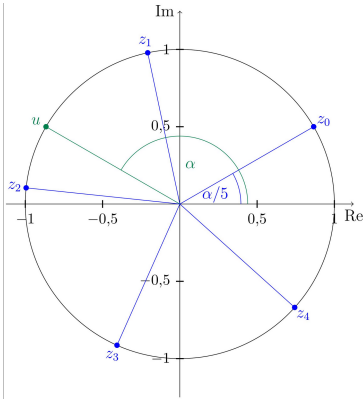
bzw.

$$z^D = 1$$

Lösungen:

$$z_k = e^{j2\pi k/D}$$

$$k = 0, \dots, D - 1$$



Alle Pole liegen auf dem Einheitskreis.

2.12 Direct Form und Canonical Form

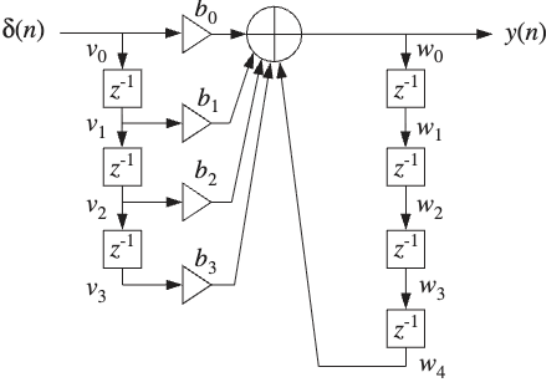


Fig. 8.1.7 Periodic generator in direct form.

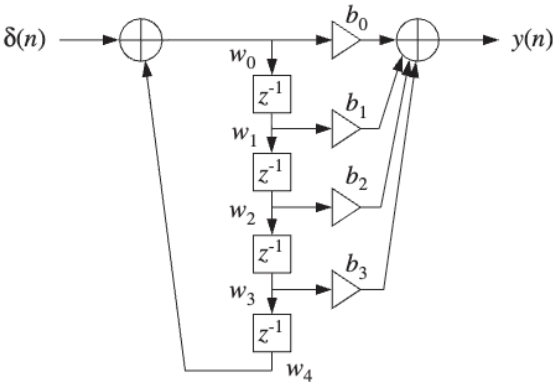


Fig. 8.1.8 Periodic generator in canonical form.

Direct Form:

- viele Delays
 - einfache Struktur
- Canonical Form:
- minimale Anzahl Delays
 - speichereffizient
 - häufig in Aufgaben verwendet

2.13 Wavetable Generator

Alternative zur Filterrealisierung:

Speichere eine Periode:

$$w[0], w[1], \dots, w[D - 1]$$

Der Zeiger läuft zyklisch:

$$y(n) = w[p(n)]$$

$$p(n + 1) = (p(n) + c) \bmod D$$

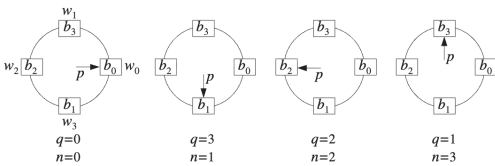


Fig. 8.1.11 Circular pointer cycles over wavetable.

Die Frequenz ergibt sich zu:

$$f = c \frac{f_s}{D}$$

2.14 Funktionsweise eines Wavetable-Generators

Eine Periode wird einmal gespeichert.

Beispiel:

$$[0, 0.7, 1, 0.7, 0, -0.7, -1, -0.7]$$

Der Pointer bewegt sich durch die Tabelle:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$$

Nach Erreichen des letzten Eintrags springt er zurück zum Anfang. Dadurch entsteht ein periodisches Signal.

2.15 Interpolation bei nichtganzzahligen Indizes

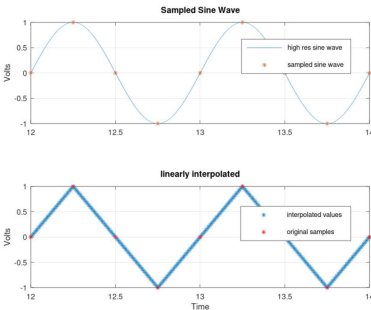
Für

$$c = 1.6$$

entstehen Indizes:

$$0, 1.6, 3.2, 4.8, \dots$$

Diese liegen zwischen den gespeicherten Samples.



Methoden:

Truncating Down:

$$k = [i]$$

Truncating Up:

$$k = \lceil i \rceil$$

Rounding:

$$k = \text{round}(i)$$

Lineare Interpolation:

$$y = (1 - \alpha)w[k] + \alpha w[k + 1]$$

2.16 Nyquist-Bedingung

Für aliasfreie Erzeugung:

$$\frac{-f_s}{2} < f < \frac{f_s}{2}$$

Mit

$$f = c \frac{f_s}{D}$$

ergibt sich:

$$-\frac{D}{2} < c < \frac{D}{2}$$

2.17 Vergleich: Filtergenerator vs. Wavetable

2.18 DTMF als Anwendung

DTMF-Töne bestehen aus zwei Sinusfrequenzen:

$$x(n) = \sin(\omega_1 n) + \sin(\omega_2 n)$$

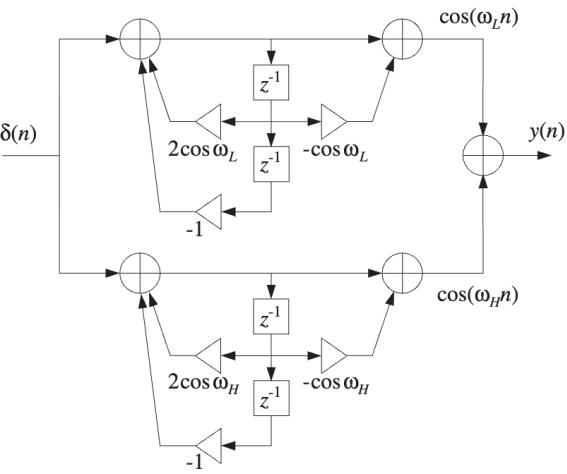


Fig. 8.1.4 DTMF tone generator.

Realisierung:

- zwei Sinusgeneratoren
- oder zwei Wavetable-Lookups

2.19 Zentrale Erkenntnisse

- Komplexe Pole erzeugen Schwingungen.
- Polwinkel bestimmt die Frequenz.
- Polradius bestimmt die Dämpfung.
- Periodische Generatoren basieren auf $1/(1 - z^{-D})$.
- Alle Pole periodischer Generatoren liegen auf dem Einheitskreis.
- Wavetable-Generatoren speichern nur eine Periode.
- Die Frequenz wird über die Schrittweite c eingestellt.
- Nichtganzzahlige Schrittweiten benötigen Interpolation.